

Comparație între algoritmul Alpha-Beta și Reinforcement Learning: De la Forță Brută la Comportament Emergent în Gomoku

1. Introducere: De ce Gomoku și de ce Reinforcement Learning?

Pentru acest proiect am ales Gomoku, o generalizare la o scară mult mai mare a clasicului „X și O” (Tic-Tac-Toe). Motivul a fost dublu: pe de o parte, este un joc cu o mecanică simplă și elegantă cu care eram familiarizat, iar pe de altă parte, din punct de vedere algoritmic este suficient de complex

Scopul principal al acestui proiect nu a fost doar construirea unui agent capabil să joace corect, ci explorarea graniței dintre logica deterministă și intuiția artificială. Prin urmare, lucrarea analizează o confruntare directă între forța brută a algoritmului clasic **Minimax (cu tăieri Alpha-Beta)** și un agent neural antrenat prin **Reinforcement Learning (RL)**.

2. Controlul Exploziei Combinatoriale: Constrângerile Mediului

În varianta originală Gomoku se joacă pe o tablă de 19x19 iar piesele se pot plasa oriunde pe tablă (într-un spațiu liber) și câștigă primul jucător care pune 5 din piesele lui în linie coloană sau diagonală (spre deosebire de 3 în X și O).

Pentru a face antrenamentul agentului de RL viabil și pentru a minimiza numărul de mutări analizate de minimax am impus următoarele constrângeri:

- i) Prima mutare trebuie făcută în centrul tablei (în varianta mea de dimensiune 20x20)
- ii) Orice altă mutare trebuie să fie adiacentă cu o piesă deja existentă pe tablă (sus, jos, stânga sau dreapta)

Această decizie limitează jocul la un singur cluster dens.

3. Arhitectura Deterministă: Construirea și Optimizarea Agentului Minimax

Am început prin a construi Environment-ul de bază, tabla de joc regulile și un mod de Rendering pentru a face jocurile vizibile, apoi am testat cu un agent random.

3.1 Euristica

Deoarece o căutare în adâncime până la finalul jocului este imposibilă în Gomoku din cauza factorului de ramificare, algoritmul trebuie să se oprească la o anumită adâncime și să evalueze poziția la acel moment. Pentru a face asta eficient am limitat analiza poziției doar la zona în care există piese pe tablă.

Am folosit un dicționar de scoruri (cheie = secvență (în format String), valoare = scor_asociat) pentru recunoașterea tiparelor (Pattern Matching – secvențele extrase din tablă erau convertite la String pentru a putea lua valoarea din dicționar). Inițial valorile asociate secvențelor (scorurile per secvență de piese) variaua liniar, fapt ce a dus la reward hacking în cadrul algoritmului de Minimax. Acesta refuza să câștige, deoarece primea per total scor mai mare din multe secvențe de câte 3 sau 4 piese în linie. Soluția a fost o scalare exponențială a scorurilor asociate secvențelor.

Tot din cauza dimensiunii tablei (20x20) trimiterea stărilor spre a fi evaluate în formă de noduri ale unui arbore nu era viabil. Ar fi însemnat ca un astfel de nod să țină o întreagă tablă (în mare parte goală), o întreagă stare. Prin urmare, am folosit o funcție de „undo” care, după ce agentul minimax face o mutare și o evaluează (și pastrează informațiile dacă aceea este cea mai buna mutare), resetează starea la momentul dinaintea mutării. În cazul unei adancimi $d=2$, în cadrul căutării, se fac 2 mutări apoi se face undo de tot atâtea ori.

3.2. Memoria Partajată (Transposition Tables) și Grupul Diedral D_4

Chiar și cu tăieri Alpha-Beta, evaluarea tablei era costisitoare. Am observat că, în timpul recursivității, Minimax-ul ajungea des în stări de joc identice, dar obținute printr-o ordine diferită a mutărilor. Mai mult, tabla de Gomoku este simetrică.

Pentru a rezolva acest bottleneck, am implementat un cache la nivel global (shared_memo), transmis ca referință tuturor agenților instanțiați. Adevărata optimizare a venit din exploatarea simetriilor geometrice. Am aplicat transformări din grupul diedral D_4 : pentru fiecare stare a tablei, generează cele 8 configurații canonice posibile (4 rotații la 90° și cele 4 ogindiri corespunzătoare). Înainte de a evalua o stare, algoritmul verifică dacă oricare dintre aceste 8 forme simetrice a fost deja calculată în trecut. Dacă da, extrage scorul direct din memorie în timp $O(1)$.

3.3. Nevoia de Viteză: Optimizarea prin Ferestre Convoluționale (Window Convolution)

Cea mai mare provocare a apărut mai târziu în proiect. Când am început antrenarea agentului de Reinforcement Learning, aveam nevoie ca Minimax-ul să joace rolul de "profesor", generând milioane de mutări pe oră. Iterarea clasică prin matrice și concatenarea string-urilor pentru a găsi tipare (ex: `_XXXX_`) era prea lentă.

Pentru a atinge viteze de producție, am rescris complet motorul de evaluare euristică, inspirându-mă din mecanismul rețelelor convoluționale. Procesul funcționează astfel:

- **Principiul de bază al ferestrelor (Sliding Windows):** În loc să analizăm linii întregi de dimensiuni variabile, am aplicat un filtru fix care „glisează” peste tablă pas cu pas. Dimensiunea optimă a acestei ferestre a fost stabilită la 7. Motivul este pur matematic: pentru a câștiga este nevoie de 5 piese consecutive, iar pentru a evalua corect potențialul ofensiv sau defensiv al unei secvențe, algoritmul are nevoie de contextul imediat de la ambele capete (1 spațiu + 5 piese + 1 spațiu = 7).

- **Encodarea matematică în Baza 3:** Manipularea structurilor de tip String în memorie (alocare, concatenare, comparare) este o operație lentă, care generează un „overhead” prea mare. Pentru a elimina acest neajuns, am privit starea celulelor la nivel pur numeric. Fiecare celulă a tablei de Gomoku poate avea exact 3 stări: spațiu gol (0), piesa jucătorului curent (1) sau piesa adversarului (2). Privind astfel lucrurile, orice secvență de 7 piese devine pur și simplu un număr scris într-un sistem de numerație cu baza 3.

- **Generarea tabelului de scoruri precalculat (Lookup Array):** Valoarea unică a oricărei ferestre de 7 celule se calculează înmulțind fiecare valoare a celulei cu puterea corespunzătoare a lui 3 ($3^0, 3^1, \dots, 3^6$). Valoarea maximă posibilă pentru o astfel de fereastră este $3^7 - 1$, adică exact 2186. Având un spațiu al stărilor atât de mic și strict delimitat (2187 de combinații posibile), am putut renunța complet la structura de Hash Map (Dicționar) în favoarea unui simplu vector (array) unidimensional alocat continuu în memorie. La inițializarea jocului (boot time), sistemul iterează de la 0 la 2186. Fiecare număr este decodificat înapoi într-o secvență de 7 stări și este evaluat lent pe baza regulilor euristice exponențiale (verificând dacă e o linie de 4 deschisă, o linie de 3 blocată la un capăt etc.). Scorul obținut este salvat în array exact la indexul corespunzător aceluși număr. Această operație costisitoare se execută o singură dată pe parcursul întregii rulări.

- **Extragerea și evaluarea în timp real ($O(1)$):** Algoritmul parcurge liniile, coloanele și diagonalele aflate strict în limitele zonei active (bounding box) și extrage secvențial ferestrele de 7 elemente. Pentru fiecare fereastră, se calculează instantaneu valoarea polinomială în baza 3 printr-un produs scalar (*dot product*) între vectorul stărilor celulelor și un vector precalculat al puterilor lui 3. Întregul proces de evaluare euristică a aceluși tipar se reduce acum la accesarea unei adrese de memorie: `scor = lookup_array[valoare_baza_3]`.

4. Arhitectura Reinforcement Learning

Odată ce am avut un motor Minimax capabil să joace și să evalueze stări la viteze mari, am putut începe antrenarea modelului de Reinforcement Learning. Am folosit algoritmul **PPO (Proximal Policy Optimization)**, o arhitectură de tip Actor-Critic, mai exact Maskable PPO. Aceasta este o optimizare potrivită care se potrivește cu frontiera de mutări valide din Environment și elimină astfel toate mutările invalide.

4.1 Arhitectura: CNN și Paralelizare

Pentru început rețeaua convoluțională folosită pentru feature extraction era una simplă care scana tabla pe un singur canal, ca o simplă imagine. Ulterior, am separat tabla pe 2 canale, unul pentru piesele agentului și unul pentru piesele adversarului. Până la final, rețeaua a ajuns să aibă 4 canale cu încă două adugate: un canal pentru frontiera de mutări valide și un canal „threat map” în care sunt encodeate pozițiile în care adversarul poate câștiga.

Pentru performanță extra, am folosit SubprocVecEnv și OMP (Open M..) pentru a paraleliza antrenarea agentului în 8 environment-uri simultan.

4.2 Curriculum Learning și Opponent Pool

Agentul a fost antrenat progresiv cu adversari progresiv mai competenți și inclusiv împotriva variantelor simple de minimax pentru a accelera procesul de învățare a defensivei. Simultan cu introducerea periodică a adversarilor statici mai buni, agentul a jucat constant și cu variante ale sale dintr-un pool dinamic.

La fiecare început de meci, mediul alege aleatoriu adversarul pe baza unor ponderi strict controlate:

- 1) Agentul a jucat exclusiv împotriva unui agent complet Random, învățând rapid să recunoască oportunitățile imediate de a face o linie de 5.
- 2) Am introdus un agent *Fast Heuristic*: dacă agentul poate să câștige o să o facă, dacă nu caută să blocheze adversarul, iar dacă nici această variantă nu este posibilă mută random.
- 3) Odată ce agentul a demonstrat coerență, am alocat majoritatea ponderii (80-90%) către un Pool de Self-Play. Sistemul salva periodic cele mai bune versiuni (snapshot-uri) ale rețelei și le forța să joace una împotriva celeilalte. Aici a început să fie vizibil fenomenul „Red Queen”: Agentul avea perioade în care câștiga foarte mult, ceea ce ducea la adăugarea adversarilor puternici în pool. Apoi agentul trebuia să câștige împotriva acestor variante mai bune ale sale fapt ce îl forța să exploreze mai mult, să se adapteze și implicit să evolueze.
- 4) Pentru a preveni „Catastrophic Forgetting”, am menținut permanent o pondere mică de agenți statici (2.5% Minimax D1, 5% Minimax D2). Oricând agentul RL uita să blocheze un atac simplu, Minimax-ul îl taxa instantaneu.

4.3 Data Augmentation

După zeci de milioane de pași de antrenament, agentul atinge scoruri excelente, dar suferea de o problemă de generalizare. Rețeaua neuronală învățase să se apere foarte bine în centru, dar, în momentul în care jocul devenea mai complex, agentul nu mai știa să blocheze mai departe de zona centrală și ar fi avut nevoie de foarte multe meciuri ca să acumuleze suficient context. Din acest motiv am aplicat aceleași transformări ca în cazul agentului Minimax (rotații la 90°, 180°, 270° și reflexii).

5. Comparația

Pentru comparația finală am pus agentul de RL (PPO) să joace 5 de meciuri împotriva agentului Minimax cu căutare la adâncime maxima 3. În această comparație am inclus următoarele metrice:

- Scorul final
- Spațiul ocupat pe disc (pentru a stoca modelul PPO / codul sursă al algoritmului Minimax)
- Timpul minim și maxim de mutare
- Timpul mediu de mutare și mediana acestuia
- Minimul și Maximul de memorie alocat în timpul meciurilor
- Media și mediana memoriei alocate
- Grafice pentru Timpul de răspuns și Memoria alocată

[DISK] PPO: 77.78 MB | Minimax: 0.0043 MB (doar script)

[FINAL] Rezultate: PPO 4 - 1 Minimax (0 Remize)

Agent	Min Timp (s)	Max Timp (s)	Avg Time (s)	Median Time (s)	Min Mem (Mb)	Max Mem (Mb)	Avg Mem (Mb)	Median Mem (Mb)
PPO	0,0040	0,0209	0,0072	0,0056	0,0223	0,3049	0,0761	0,0293
Minimax	0,0423	2,5941	0,8370	0,6257	0,0764	1,7205	0,5361	0,3889

După cum se observă din datele agregate pe cele 5 de meciuri, agentul minimax este semnificativ mai lent și ocupă mai multă memorie (în special din cauza optimizării de timp cu transposition tables), pe când agentul de Reinforcement Learning PPO are timp de răspuns minim și memorie alocată aproape 0. Pețul plătit pentru agentul PPO este, în schimb, spațiu pe disc și semnificativ mai mult timp de antrenare (estimez undeva la 120 de ore timp de antrenare total)

Datele complete: [#Anexa_date_complete](#)

6. Concluzii

Acest proiect a explorat granița dintre calculul determinist, bazat pe forță brută, și intuiția artificială dezvoltată prin Reinforcement Learning. În urma implementării și evaluării celor doi agenți — Minimax (Alpha-Beta Pruning) și PPO (Proximal Policy Optimization) — am desprins următoarele concluzii fundamentale:

Compromisul dintre Antrenament și Inferență: În timp ce Minimax nu necesită antrenament, acesta devine foarte lent pe măsură ce adâncimea de căutare crește, consumând resurse computaționale semnificative la fiecare mutare. În contrast, agentul PPO a necesitat un cost computațional uriaș în etapa de antrenament, dar oferă un timp de răspuns aproape instantaneu și constant în faza de joc (inferență), fiind ideal pentru aplicații în timp real.

Capacitatea de Generalizare vs. Memorare: PPO a demonstrat o capacitate superioară de generalizare, reușind să învingă adversari tactici (precum Minimax D3) pe care nu i-a întâlnit direct în antrenament. Totuși, performanța sa este direct dependentă de calitatea criticului și de echilibrul dintre explorare și exploatare.

7. Bibliografie

- Raffin, A. et al. (2021). *Stable Baselines3: Reliable Reinforcement Learning Implementations*. GitHub repository. Disponibil la: <https://stable-baselines3.readthedocs.io/>
- Farama Foundation. *Gymnasium: A standard API for reinforcement learning*. Disponibil la: <https://gymnasium.farama.org/>
- [Nicholas Renotte] (2022). *Reinforcement Learning in 3 hours | Full Course using Python*. https://www.youtube.com/watch?v=Mut_u40Sqz4

Anexa A: Loguri complete și grafice pentru antrenament

Joc	Agent	Min Time (s)	Max Time (s)	Avg Time (s)	Median Time (s)	Min Mem (Mb)	Max Mem (Mb)	Avg Mem (Mb)	Median Mem (Mb)
1	PPO	0,0040	0,0088	0,0052	0,0047	0,0220	0,0288	0,0229	0,0224
1	Minimax	0,1303	3,3702	1,0412	0,7669	0,0983	2,1564	0,7068	0,4639
2	PPO	0,0042	0,0670	0,0126	0,0067	0,0222	0,5402	0,1413	0,0223
2	Minimax	0,0659	1,8260	0,8396	0,7367	0,1989	0,9515	0,5203	0,5180
3	PPO	0,0040	0,0090	0,0060	0,0059	0,0223	0,0378	0,0274	0,0229
3	Minimax	0,0066	2,1335	0,9249	0,8997	0,0294	1,1960	0,5047	0,4547
4	PPO	0,0041	0,0110	0,0058	0,0045	0,0224	0,8769	0,1559	0,0436
4	Minimax	0,0022	2,2298	0,7984	0,6490	0,0220	2,7880	0,6743	0,4690
5	PPO	0,0037	0,0089	0,0066	0,0064	0,0224	0,0410	0,0332	0,0352
5	Minimax	0,0064	3,4112	0,5810	0,0762	0,0333	1,5104	0,2745	0,0387

[DISK] PPO: 77.78 MB | Minimax: 0.0043 MB (doar script)

[FINAL] Rezultate: PPO 4 - 1 Minimax (0 Remize)

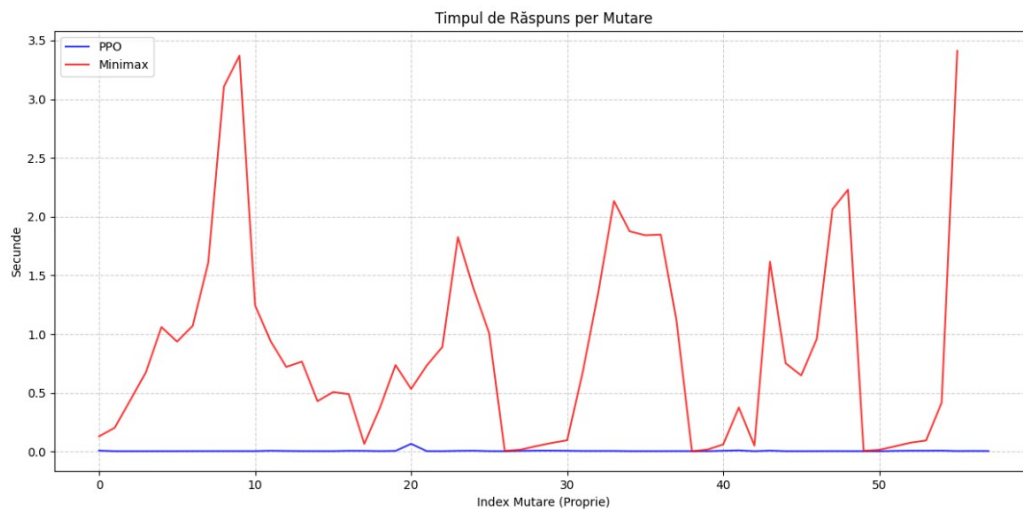


Figure 1: Timpul de gândire pentru mutare

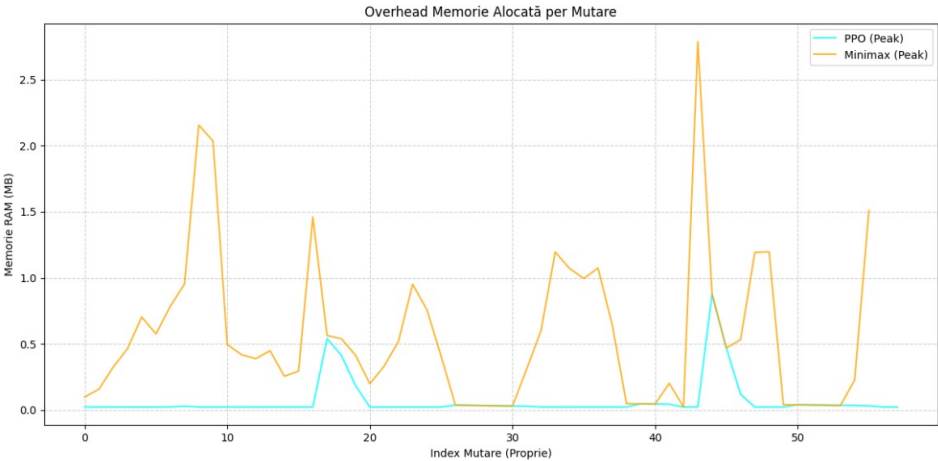


Figure 2: Memorie alocată la mutare

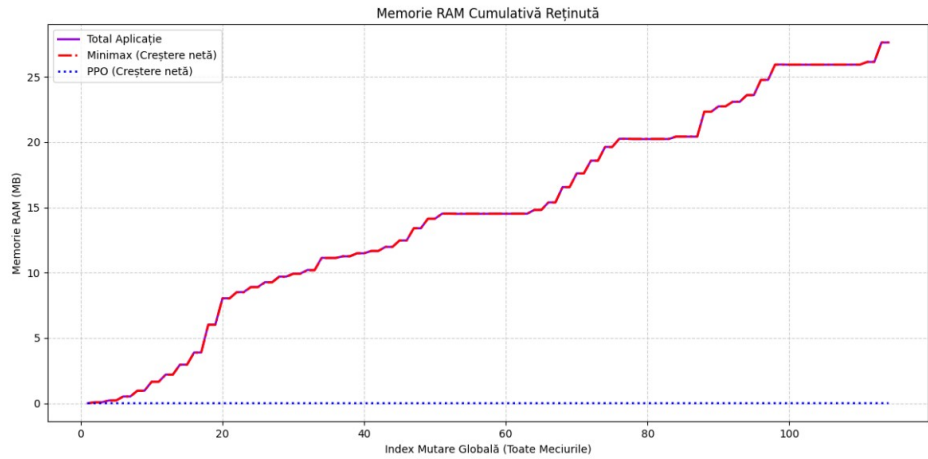


Figure 3: Memoria Ram cumulativa consumata

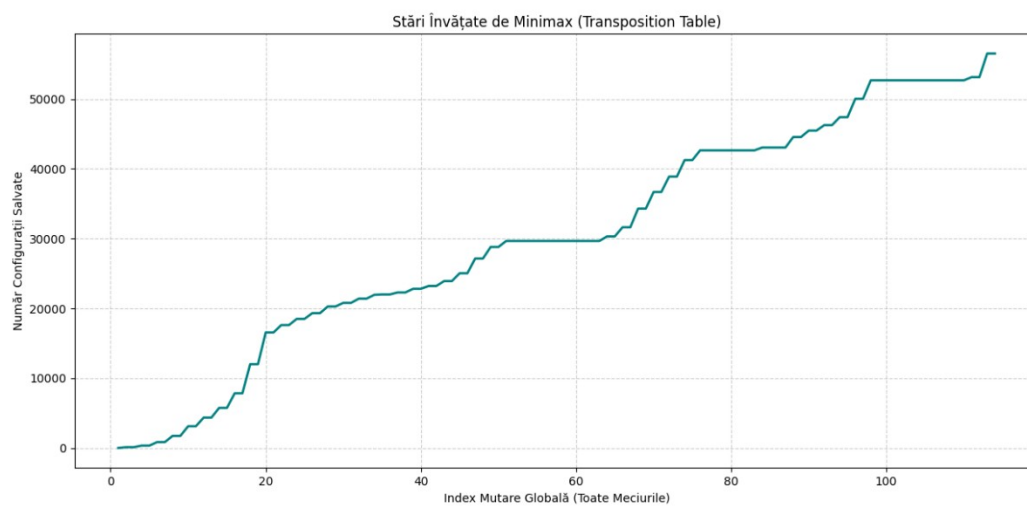


Figure 4: Numărul de simetrii stocate de Minimax